

Rasoi Sakhi

SYSTEM DOCUMENTATION & LAUNCH GUIDE

A comprehensive operations manual for developers, clients, and administrators.

Version: 1.0.0 (Production-Ready)

Date: June 2026

Primary Deployment: Vercel Serverless Hosting

Database & Storage: Supabase PostgreSQL & Bucket
Storage

1. System Overview

Rasoi Sakhi is a premium web platform built specifically for ordering professionally cleaned, washed, and cut ready-to-cook vegetables. The website operates as a Single Page Application (SPA) on the frontend for ultra-fast, premium-feel transitions, backed by an Express API deployed to Vercel Serverless Functions and a Supabase database.

Core Tech Stack

Component	Technology	Purpose
Frontend	HTML5, Vanilla CSS3, Javascript (ES6)	Responsive user interfaces, order flow, admin settings panel.
Backend	Node.js, Express framework	Serverless API handling order creation, setting controls, and webhook callbacks.
Database	Supabase (PostgreSQL)	Persistent storage for products, orders, settings, users, and push keys.
File Storage	Supabase Storage Buckets	Image hosting for custom product photos.
Payments	Razorpay SDK (Live)	Secure online credit/debit card, netbanking, and UPI checkout flow.

2. Razorpay Live Integration & Webhooks

Online payment verification uses a dual client-server flow combined with Razorpay webhooks to guarantee order status security and prevent payment bypass hacks.

A. Payment Verification Flow

1. **Order Creation:** When a customer clicks checkout, the server calls Razorpay to generate a unique `Razorpay Order ID` (prefixed with `order\_`).
2. **Database Mapping:** The server maps this `Razorpay Order ID` to our local database order ID in a secure database table (`payment\_mappings`) before returning the details to the user.
3. **Checkout Window:** The frontend initializes the Razorpay SDK checkout overlay. The user selects UPI, Card, or Netbanking.
4. **Webhook Callback (The Automator):** Once the payment is successfully completed, Razorpay issues a server-to-server HTTP POST request to our webhook endpoint `/api/payments/webhook`.
5. **Signature Verification:** The server reads the raw binary body (`req.rawBody`) and computes an HMAC-SHA256 signature using the secret configured in `.env`. If the computed signature matches the header `x-razorpay-signature`, the payload is verified as genuine.
6. **Status Progression:** The server checks the local database mapping table using the received `order\_id`, changes the order status from **Payment Pending** to **Payment Received**, and pushes the order details to the Google Sheets webhook.

B. Setting Up Webhooks in Razorpay Dashboard

To automate status updates, the client must configure the webhook in their Razorpay account:

- **Webhook URL:** `https://<your-vercel-domain>/api/payments/webhook`
- **Secret:** `rasoisakhi_live_webhook_secret_secure_2026`
- **Active Event:** `order.paid`

Webhook Mismatch Warning

If the signature fails validation, the status will remain **Payment Pending**. Ensure that the `RAZORPAY_WEBHOOK_SECRET` matches exactly between the Razorpay Dashboard and the Vercel/environment configuration.

3. Delivery Slot Logic & Cut-offs

Orders are governed by strict scheduling constraints to ensure the kitchen has time to wash, sanitize, and cut the vegetables before delivery. The server enforces these slots, and the frontend automatically updates the dropdown labels and disables past slots.

Active Delivery Slots Table

Slot name	Delivery hours	Order cut-off rule	Availability label format
Morning	6:00 AM – 7:30 AM	Order must be placed the previous night after 6:00 PM	Morning – 6:00 AM to 7:30 AM (order tonight after 6 PM)
Afternoon	11:00 AM – 12:30 PM	Order must be placed today before 10:00 AM	Afternoon – 11:00 AM to 12:30 PM (order by 10 AM)
Evening	5:00 PM – 6:30 PM	Order must be placed today before 4:00 PM	Evening – 5:00 PM to 6:30 PM (order by 4 PM)

How the Logic Behaves:

- **If current time is 11:00 AM (Today):** The Afternoon and Evening slots are closed (it is past 10:00 AM, and evening is not yet 6:00 PM for tomorrow morning's slot). The Morning slot for tomorrow is also closed since it's not yet 6:00 PM. They will show as **– Not Available**.
- **If current time is 6:30 PM (Tonight):** The Morning slot is unlocked since it is after 6:00 PM. The Afternoon and Evening slots are closed.
- **If a slot is closed:** It is disabled (greyed out) in the checkout dropdown list, ensuring customers cannot place an order for a delivery time that the kitchen cannot fulfill.

4. Progressive Web App (PWA) & Push Notifications

Rasoi Sakhi supports instant mobile push notifications for the admin, alerting them the second an order changes to "Payment Received". This is built using standard Web Push protocols (VAPID) and Service Workers.

A. The iOS Safari Issue & Workaround

By default, Apple's iOS Safari disables push notification APIs to prevent websites from spamming users. The only way to activate them is to use PWA features:

1. **Install as App:** The user must install the website to their Home Screen. We provide a `manifest.json` linked in the HTML head to let Safari register it as a standalone app.
2. **Add to Home Screen:** Open the site on an iPhone in Safari, tap the **Share** button, and tap **"Add to Home Screen"**.
3. **Launch & Accept:** Open the newly installed app from the Home Screen, go to the Admin Settings panel, and tap **"Enable Notifications"**. Because the app runs in standalone PWA mode, iOS Safari unlocks `PushManager` and shows the permission popup.

B. How the Push flow works

1. The admin enables notifications, producing a subscription object containing a unique browser endpoint.
2. The subscription is saved to the Supabase database.
3. When a payment is captured via the webhook, the server queries active push subscriptions and triggers web-push notifications using the server VAPID keys.
4. The browser's service worker (`sw.js`) intercepts the event in the background and shows the alert on the admin's device.

5. System Settings & Configuration

The Admin Dashboard has a settings tab that saves rules directly into the Supabase database. These sync to the client app in real-time.

A. Allowed Delivery Pincodes

To restrict delivery to valid regions in Bharuch, the admin can configure multiple pincodes. Enter them as a ****comma-separated list****:

```
392011, 392012, 392015
```

The frontend validates checkout entries by converting the input to an array and checking if the customer's pincode is supported. If a customer enters a pincode not on the list (e.g. `392001`), checkout will block and show supported delivery options.

B. WhatsApp Number Configuration

Enter the mobile number with country code, but without spaces or symbols (e.g. `919099113823`). When a customer completes checkout, the system compiles a formatted order details message and redirects them to WhatsApp to start a conversation with the kitchen.

C. Delivery Charges & Free Delivery

- **Base Delivery Charge:** The fee added to checkout (e.g., ₹30).
- **Free Delivery Threshold:** The subtotal amount above which delivery fee is set to ₹0 (e.g., ₹299).

6. Troubleshooting & Maintenance

A reference checklist for developers if something goes wrong in production.

1. Payments are completed but orders stay "Payment Pending"

- **Check Webhook Logs:** Check the Razorpay Dashboard to see if webhook HTTP requests are returning success status (200).
- **Check Secret Mismatch:** Ensure that the environment variable `RAZORPAY\_WEBHOOK\_SECRET` on Vercel matches the secret entered on the Razorpay Webhooks dashboard.
- **Supabase Connection:** Ensure Supabase service role keys have not expired. Check server start logs to ensure database connections are active.

2. "Invalid Webhook Signature" errors on the server

- This happens if Express parses the body before verification. Ensure that `req.rawBody` is used. We have implemented raw body extraction in our body parser:

```
app.use(express.json({  
  limit: '10mb',  
  verify: (req, res, buf) => { req.rawBody = buf; }  
}));
```

3. Uploaded product images are slow to load

- **WebP Auto-Conversion:** Ensure `sharp` is present in your production dependencies (NOT devDependencies) in `package.json`. The Express server is programmed to intercept base64 uploads, compress them, convert them to WebP format, and rename them to `.webp` before sending to Supabase storage. If `sharp` fails to load, uploads will crash. Check Vercel function log errors.

4. Google Sheets is not updating

- **Quota limits:** Google Sheets Apps Script has a daily quota on URL requests. Check Apps Script logs in the Google Drive console.
- **Setting URL check:** Make sure the Apps Script execution URL is saved correctly in the Admin Dashboard Settings page.